

Apartment Corp — Rent Comparables API

Developer Reference & Integration Guide

Project: Apartment Corp Rent Comps API (`rentcomps-api`)

Base URL (Production): `https://rentcompsapi-bjuoqf2s.manus.space`

Base URL (Development): `http://localhost:3000`

Stack: Node.js · Express 4 · tRPC 11 · React 19 · Drizzle ORM · TiDB (MySQL-compatible)

Authentication: Manus OAuth 2.0 (session cookie) — see Section 3

Last Updated: July 1, 2026

Table of Contents

1. Overview
 2. Architecture Summary
 3. Authentication
 4. REST Endpoints
 5. tRPC Procedures
 6. Data Models
 7. Database Schema
 8. Error Handling
 9. Admin Operations
 10. Scheduled Jobs
 11. Quick-Start Examples
 12. Environment Variables
-

1. Overview

The Rent Comparables API serves as the data backbone for Apartment Corp's internal rent analysis platform. It stores, parses, and exposes rent comparable data for all 42 Apartment Corp properties across the United States. The system ingests data from a manually maintained Excel workbook (`RentComps_FINAL.xlsx`) and makes it available through both a public REST API and a type-safe tRPC interface for the React frontend.

The platform supports two consumer types:

- **Internal frontend (React):** Uses the tRPC client for fully type-safe, end-to-end queries and mutations with no manual type definitions required.
- **External integrations (PropertyMax, reporting tools, etc.):** Use the plain REST endpoints documented in Section 4, which return standard JSON and require no special client library.

2. Architecture Summary

Browser (React 19 + Vite)

— /api/trpc/*	← tRPC procedures (type-safe, frontend use)
— /api/rentcomps	← REST: all buildings with full comp data
— /api/rentcomps/:id	← REST: single building by buildingId
— /api/oauth/*	← Manus OAuth flow
— /api/scheduled/*	← Heartbeat cron endpoints

Server (Express 4 + tsx)

— server/routers.ts	← All tRPC procedures
— server/db.ts	← Database query helpers
— server/_core/	← OAuth, context, LLM, storage, map, etc.
— drizzle/schema.ts	← Database table definitions

Database: TiDB Cloud (MySQL-compatible)

— users
— buildings
— comps
— workbook_uploads

3. Authentication

3.1 Manus OAuth Flow

The API uses Manus OAuth 2.0 for user authentication. The session is stored in a signed HTTP-only cookie.

Login flow:

1. Frontend calls `getLoginUrl(returnPath?)` from `client/src/const.ts` — this generates the OAuth redirect URL with the app's `origin` encoded in the `state` parameter.
2. User is redirected to the Manus login portal at `https://manus.im`.
3. After login, Manus redirects to `GET /api/oauth/callback?code=...&state=...`.
4. The server exchanges the code for a user token, creates or updates the user record, sets a signed session cookie, and redirects the browser back to the return path.

Session cookie: `manus_session` (HTTP-only, SameSite=Lax, signed with `JWT_SECRET`)

3.2 Access Levels

Level	How to obtain	What it unlocks
Public	No authentication required	<code>GET /api/rentcomps</code> , <code>GET /api/rentcomps/:id</code> , <code>trpc.rentcomps.list</code> , <code>trpc.rentcomps.getById</code> , <code>trpc.rentcomps.listFull</code>
Authenticated User	Valid session cookie	<code>trpc.auth.me</code> , <code>trpc.auth.logout</code>
Admin	User record with <code>role = 'admin'</code> in database	<code>trpc.rentcomps.uploadWorkbook</code> , <code>trpc.rentcomps.uploadHistory</code>

To promote a user to admin, run the following SQL via the Database panel or `webdev_execute_sql`:

```
UPDATE users SET role = 'admin' WHERE email = 'brandon@apartmentcorp.com'
```

4. REST Endpoints

All REST endpoints return JSON. No API key is required for the public endpoints.

GET /api/rentcomps

Returns all buildings with their full comparable data — affordable comps, market-rate comps, FMR benchmark, and market average.

Authentication: None required

Rate limit: None (internal use)

Response shape:

```
{
  "success": true,
  "count": 42,
  "lastRefreshed": "2026-04-30T00:00:00.000Z",
  "data": [
    {
      "buildingId": "21 - Boca Ciega",
      "name": "Boca Ciega",
      "address": "5301 28th Ave S",
      "city": "Gulfport",
      "state": "MS",
      "county": "Harrison County",
      "msa": "Gulfport-Biloxi MSA",
      "program": "LIHTC / Section 8",
      "totalUnits": 100,
      "yearBuilt": 1970,
      "currentRents": { "2BD": 1020, "3BD": 1150 },
      "marketAvg": { "2BD": 1380, "3BD": 1507 },
      "fmrByBed": { "2BD": 1210, "3BD": 1540 },
      "lastRefreshed": "2026-04-30T00:00:00.000Z",
      "comps": {
        "affordable": [ ...comp objects ],
        "market": [ ...comp objects ],
        "fmr": { ...fmr benchmark object },
        "marketAverage": { ...market average object }
      }
    }
  ]
}
```

Example request:

```
curl https://rentcompsapi-bjuoqf2s.manus.space/api/rentcomps
```

GET /api/rentcomps/:buildingId

Returns full comp data for a single building. The `buildingId` must match the sheet name format used in the workbook (e.g., `21 - Boca Ciega`). URL-encode the value when it contains spaces or special characters.

Authentication: None required

Path parameter:

Parameter	Type	Example
<code>buildingId</code>	string	<code>21%20-%20Boca%20Ciega</code>

Success response (200):

```
{
  "success": true,
  "data": {
    "buildingId": "21 - Boca Ciega",
    "name": "Boca Ciega",
    ...full building object (same shape as items in /api/rentcomps)
  }
}
```

Not found response (404):

```
{
  "success": false,
  "error": "Building \"21 - Boca Ciega\" not found"
}
```

Example requests:

```
# URL-encoded buildingId
curl "https://rentcompsapi-bjuoqf2s.manus.space/api/rentcomps/21%20-%20Bo

# Using --data-urlencode with curl
curl -G "https://rentcompsapi-bjuoqf2s.manus.space/api/rentcomps" \
  --data-urlencode "buildingId=21 - Boca Ciega"
```

Comp Object Shape

Each item in `comps.affordable` and `comps.market` follows this structure:

```
{
  "propertyName": "Sherbondy Villas",
  "address": "200 S Perviz Ave",
  "city": "Opa Locka",
  "state": "FL",
  "distanceMiles": "1.20",
  "yearBuilt": 1967,
  "totalUnits": 32,
  "program": null,
  "competitionLevel": "HIGH",
  "rents": {
    "1BD": { "asking": null, "actual": 1400 },
    "2BD": { "asking": null, "actual": null }
  }
}
```

5. tRPC Procedures

The tRPC API is mounted at `/api/trpc`. All procedures are called via the tRPC client in the React frontend. External callers can also use tRPC's HTTP batch protocol directly, though the REST endpoints in Section 4 are recommended for external integrations.

5.1 Auth Procedures

Procedure	Type	Auth	Description
<code>trpc.auth.me</code>	query	Public	Returns the current user object, or <code>null</code> if not logged in
<code>trpc.auth.logout</code>	mutation	Public	Clears the session cookie and returns <code>{ success: true }</code>

Frontend usage:

```
const { data: user } = trpc.auth.me.useQuery();
const logout = trpc.auth.logout.useMutation();

// Check if logged in
if (user) {
  console.log(user.name, user.role); // "Brandon Rose", "admin"
}

// Log out
await logout.mutateAsync();
```

5.2 Rent Comps Procedures

Procedure	Type	Auth	Description
<code>trpc.rentcomps.list</code>	query	Public	Returns summary list of all buildings (no comp detail)
<code>trpc.rentcomps.getById</code>	query	Public	Returns full comp data for one building
<code>trpc.rentcomps.listFull</code>	query	Public	Returns all buildings with full comp data
<code>trpc.rentcomps.uploadWorkbook</code>	mutation	Admin only	Parses and ingests a new Excel workbook
<code>trpc.rentcomps.uploadHistory</code>	query	Admin only	Returns log of all workbook uploads

`trpc.rentcomps.list`

Returns a lightweight summary of all buildings — useful for populating dropdowns or index pages.

```
const { data: buildings } = trpc.rentcomps.list.useQuery();  
// buildings[0] = { buildingId, name, city, state, program, totalUnits, c
```

`trpc.rentcomps.getById`

```
const { data: building } = trpc.rentcomps.getById.useQuery({  
  buildingId: "21 - Boca Ciega"  
});  
// Returns full building response including all comps
```

Input schema:

```
z.object({ buildingId: z.string() })
```

Throws: `TRPCError { code: "NOT_FOUND" }` if the buildingId does not exist.

trpc.rentcomps.uploadWorkbook

Admin-only mutation that accepts a base64-encoded Excel workbook, stores it in S3, parses all building sheets, and upserts the data into the database.

```
const upload = trpc.rentcomps.uploadWorkbook.useMutation();

// Convert file to base64 first
const fileBase64 = btoa(String.fromCharCode(...new Uint8Array(fileBuffer)));

await upload.mutateAsync({
  filename: "RentComps_FINAL_v5.xlsx",
  fileBase64: fileBase64
});
// Returns: { success: true, buildingsParsed: 42, compsParsed: 504 }
```

Input schema:

```
z.object({
  filename: z.string(),
  fileBase64: z.string() // base64-encoded .xlsx file content
})
```

Throws: `TRPCError { code: "FORBIDDEN" }` if the user is not an admin.

Throws: `TRPCError { code: "INTERNAL_SERVER_ERROR" }` if parsing fails.

trpc.rentcomps.uploadHistory

```
const { data: history } = trpc.rentcomps.uploadHistory.useQuery();  
// Returns array of workbook_uploads records sorted by createdAt desc
```

6. Data Models

Building Summary (from trpc.rentcomps.list)

```
{  
  buildingId: string;           // e.g. "21 - Boca Ciega"  
  name: string;                 // e.g. "Boca Ciega"  
  city: string;  
  state: string;  
  program: string;              // e.g. "LIHTC / Section 8"  
  totalUnits: number;  
  currentRents: Record<string, number>; // { "2BD": 1020, "3BD": 1150 }  
  marketAvg: Record<string, number>;    // { "2BD": 1380, "3BD": 1507 }  
  lastRefreshed: Date | null;  
}
```

Full Building Response (from `trpc.rentcomps.getById` / REST)

```
{
  buildingId: string;
  name: string;
  address: string | null;
  city: string | null;
  state: string | null;
  county: string | null;
  msa: string | null;
  program: string | null;
  totalUnits: number | null;
  yearBuilt: number | null;
  currentRents: Record<string, number>;
  marketAvg: Record<string, number>;
  fmrByBed: Record<string, number>;    // { "2BD": 1210, "3BD": 1540 }
  lastRefreshed: Date | null;
  comps: {
    affordable: CompItem[];
    market: CompItem[];
    fmr: CompItem | null;
    marketAverage: CompItem | null;
  };
}
```

ComplItem

```
{
  propertyName: string;
  address: string | null;
  city: string | null;
  state: string | null;
  distanceMiles: string | null; // decimal string, e.g. "1.20"
  yearBuilt: number | null;
  totalUnits: number | null;
  program: string | null;
  competitionLevel: string | null; // "HIGH" | "MODERATE-HIGH" | "LOW-MODI
  rents: Record<string, { asking: number | null; actual: number | null }>
}
```

7. Database Schema

The database runs on TiDB Cloud (fully MySQL-compatible). All timestamps are stored in UTC.

users table

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Internal row ID
openId	VARCHAR(64) UNIQUE	Manus OAuth user identifier
name	TEXT	Display name
email	VARCHAR(320)	User email
loginMethod	VARCHAR(64)	OAuth provider
role	ENUM('user','admin')	Default: user
createdAt	TIMESTAMP	Auto-set on insert
updatedAt	TIMESTAMP	Auto-updated on change
lastSignedIn	TIMESTAMP	Updated on each login

buildings table

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Internal row ID
buildingId	VARCHAR(64) UNIQUE	Sheet name, e.g. 21 - Boca Ciega
name	VARCHAR(128)	Short name, e.g. Boca Ciega
address	TEXT	Street address
city	VARCHAR(64)	
state	VARCHAR(32)	
county	VARCHAR(64)	
msa	VARCHAR(128)	Metro Statistical Area
program	VARCHAR(64)	e.g. LIHTC / Section 8
totalUnits	INT	
yearBuilt	INT	
fmr2bd	DECIMAL(10,2)	2BD Fair Market Rent
fmr3bd	DECIMAL(10,2)	3BD Fair Market Rent
fmrOther	JSON	{ "1BD": 900, "4BD": 1600 }
currentRents	JSON	{ "2BD": 1020, "3BD": 1150 }
marketAvg	JSON	{ "2BD": 1380, "3BD": 1507 }
lastRefreshed	TIMESTAMP	Date of most recent workbook upload
createdAt	TIMESTAMP	
updatedAt	TIMESTAMP	

comps table

Column	Type	Notes
id	INT AUTO_INCREMENT PK	
buildingDbId	INT	FK → buildings.id
compType	ENUM	affordable market fmr market_avg
sortOrder	INT	Display order within type group
propertyName	VARCHAR(128)	
address	TEXT	
city	VARCHAR(64)	
state	VARCHAR(32)	
distanceMiles	DECIMAL(5,2)	
yearBuilt	INT	
totalUnits	INT	
program	VARCHAR(64)	e.g. LIHTC, Section 8
competitionLevel	VARCHAR(32)	HIGH, MODERATE-HIGH, LOW-MODERATE, BENCHMARK
rents	JSON	{ "2BD": { "asking": 1500, "actual": 1450 } }
createdAt	TIMESTAMP	
updatedAt	TIMESTAMP	

workbook_uploads table

Column	Type	Notes
id	INT AUTO_INCREMENT PK	
filename	VARCHAR(256)	Original filename
storageKey	VARCHAR(512)	S3 object key
uploadedBy	VARCHAR(64)	Manus openId of uploader
buildingsParsed	INT	Count of buildings upserted
compsParsed	INT	Count of comp rows upserted
status	ENUM	pending processing success error
errorMessage	TEXT	Populated on parse failure
createdAt	TIMESTAMP	

8. Error Handling

REST Endpoints

All REST endpoints return a consistent JSON envelope:

```
{ "success": true, "data": { ... } }           // 200 OK
{ "success": false, "error": "message" }       // 4xx / 5xx
```

HTTP Status	Meaning
200	Success
404	Building not found
500	Internal server error (parse failure, DB error, etc.)

tRPC Procedures

tRPC errors follow the standard tRPC error shape. Common error codes:

Code	HTTP equivalent	When thrown
NOT_FOUND	404	getById with unknown buildingId
FORBIDDEN	403	Admin procedure called by non-admin user
UNAUTHORIZED	401	Protected procedure called without session
INTERNAL_SERVER_ERROR	500	Workbook parse failure, DB error

Frontend error handling example:

```
const upload = trpc.rentcomps.uploadWorkbook.useMutation({
  onError: (err) => {
    if (err.data?.code === 'FORBIDDEN') {
      toast.error("Admin access required.");
    } else {
      toast.error(`Upload failed: ${err.message}`);
    }
  }
});
```

9. Admin Operations

Uploading a New Workbook

The recommended workflow for refreshing rent comp data is:

1. Open the Admin Dashboard at `https://rentcompsapi-bjuoqf2s.manus.space/admin`.
2. Log in with your Apartment Corp Manus account (admin role required).
3. Click **Upload Workbook** and select the latest `RentComps_FINAL.xlsx` file.
4. The system will parse all building sheets, upsert the data, and display a summary of buildings and comps processed.

Alternatively, call the tRPC mutation directly:

```
// Node.js script example
import { createTRPCClient, httpBatchLink } from '@trpc/client';
import type { AppRouter } from './server/routers';
import fs from 'fs';

const client = createTRPCClient<AppRouter>({
  links: [httpBatchLink({
    url: 'https://rentcompsapi-bjuoqf2s.manus.space/api/trpc',
    headers: { Cookie: 'manus_session=YOUR_SESSION_COOKIE' }
  })]
});

const fileBuffer = fs.readFileSync('RentComps_FINAL.xlsx');
const fileBase64 = fileBuffer.toString('base64');

const result = await client.rentcomps.uploadWorkbook.mutate({
  filename: 'RentComps_FINAL.xlsx',
  fileBase64
});

console.log(`Parsed ${result.buildingsParsed} buildings, ${result.compsPa
```

Promoting a User to Admin

```
UPDATE users SET role = 'admin' WHERE email = 'brandon@apartmentcorp.com'
```

Run this via the Database panel in the Manus Management UI, or via `webdev_execute_sql` .

10. Scheduled Jobs

The API includes one scheduled heartbeat job, triggered by Manus's built-in cron system.

Monthly Refresh Reminder

Endpoint: `POST /api/scheduled/monthly-refresh-reminder`

Schedule: 10th of every month

Authentication: Manus cron token (automatically injected by the scheduler)

When triggered, this endpoint sends a push notification to the project owner (Brandon Rose) reminding them to upload the latest workbook. The notification includes a direct link to the Admin Dashboard.

Notification content:

 Rent Comps — Monthly Refresh Reminder

It's the 10th of the month. Please upload the latest RentComps_FINAL.xlsx workbook to the Admin Dashboard to refresh the rent comp data for all 42 buildings.

The endpoint returns `403 Forbidden` if called without a valid cron token, preventing unauthorized triggering.

11. Quick-Start Examples

JavaScript / Fetch (External Integration)

```
// Get all buildings
const response = await fetch(
  'https://rentcompsapi-bjuoqf2s.manus.space/api/rentcomps'
);
const { data: buildings } = await response.json();

buildings.forEach(b => {
  console.log(`${b.name} (${b.city}, ${b.state})`);
  console.log(`  Current 2BD: ${b.currentRents['2BD']}`);
  console.log(`  Market Avg 2BD: ${b.marketAvg['2BD']}`);
});
```

```
// Get a single building
const res = await fetch(
  'https://rentcompsapi-bjuoqf2s.manus.space/api/rentcomps/21%20-%20Boca%'
);
const { data: building } = await res.json();

console.log(`${building.comps.market.length} market-rate comps`);
building.comps.market.forEach(c => {
  console.log(`  ${c.propertyName}: ${JSON.stringify(c.rents)}`);
});
```

Python (Reporting / PropertyMax Integration)

```
import requests
import json

BASE_URL = "https://rentcompsapi-bjuoqf2s.manus.space"

# Fetch all buildings
resp = requests.get(f"{BASE_URL}/api/rentcomps")
resp.raise_for_status()
payload = resp.json()

print(f"Total buildings: {payload['count']}")
for bldg in payload['data']:
    curr = bldg.get('currentRents', {})
    mkt = bldg.get('marketAvg', {})
    for bed in curr:
        if bed in mkt:
            diff = curr[bed] - mkt[bed]
            print(f"{bldg['name']} {bed}: current=${curr[bed]:,} market=
```

React (Frontend — tRPC)

```
import { trpc } from '@lib/trpc';

function BuildingList() {
  const { data: buildings, isLoading } = trpc.rentcomps.list.useQuery();

  if (isLoading) return <p>Loading...</p>;

  return (
    <ul>
      {buildings?.map(b => (
        <li key={b.buildingId}>
          <strong>{b.name}</strong> - {b.city}, {b.state} ({b.totalUnits}
        </li>
      ))}
    </ul>
  );
}
```

12. Environment Variables

The following environment variables are automatically injected by the Manus platform. Developers do not need to set these manually.

Variable	Purpose
<code>DATABASE_URL</code>	TiDB Cloud MySQL connection string
<code>JWT_SECRET</code>	Session cookie signing secret
<code>VITE_APP_ID</code>	Manus OAuth application ID
<code>OAUTH_SERVER_URL</code>	Manus OAuth backend base URL
<code>VITE_OAUTH_PORTAL_URL</code>	Manus login portal URL (frontend)
<code>OWNER_OPEN_ID</code>	Owner's Manus OpenID (for notifications)
<code>OWNER_NAME</code>	Owner's display name
<code>BUILT_IN_FORGE_API_URL</code>	Manus built-in API base URL (server-side)
<code>BUILT_IN_FORGE_API_KEY</code>	Bearer token for Manus built-in APIs (server-side)
<code>VITE_FRONTEND_FORGE_API_KEY</code>	Bearer token for Manus built-in APIs (frontend)
<code>VITE_FRONTEND_FORGE_API_URL</code>	Manus built-in API URL (frontend)

Important: Never commit `.env` files or hardcode these values in source code. All secrets are managed through the Manus Secrets panel.

Appendix: Building ID Reference

The `buildingId` field matches the Excel sheet tab name exactly. A selection of active building IDs:

buildingId	Name	City	State	Units
21 - Boca Ciega	Boca Ciega	Gulfport	MS	100
13 - Jefferson	Jefferson	Monticello	FL	60
18 - Opa Locka	Opa Locka 135th St	Opa Locka	FL	65
14 - Macedonia	Macedonia	Panama City	FL	100
48 - Coral Village	Coral Village	Cape Coral	FL	72
15 - Holiday	Holiday Apts	Natchez	MS	115
16 - Cumberland	Cumberland	—	—	—
59 - Walnut Hill	Walnut Hill	—	—	—
29 - River Pointe	River Pointe	—	—	—

Use `GET /api/rentcomps` to retrieve the full list of all 42 active building IDs.

*Document prepared by Manus AI for Apartment Corp internal developer use.
For questions, contact Brandon Rose — brandon@apartmentcorp.com*